

Desarrollo entorno Cliente

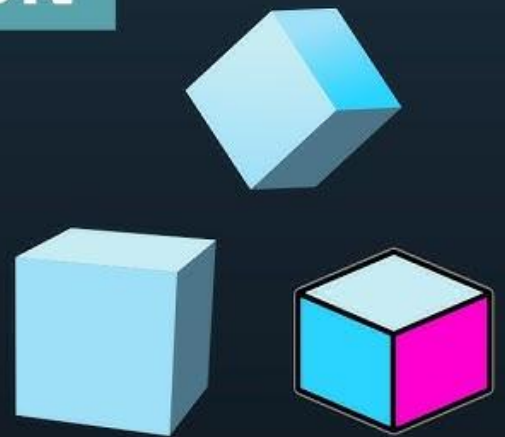
Programación Orientada a Objetos JavaScript

PROGRAMACIÓN

ORIENTADA

A OBJETOS

JS



PabloSG

Índice

1.Título del tema	2
2.Explicación detallada del contenido	2
2.1 Qué es la POO en JavaScript	2
2.1.1 Class	2
2.1.2 Constructor	2
2.1.3 Getters y Setters	3
2.1.4 Método	3
2.1.5 Sobrescritura de métodos heredados	3
2.1.6 Statics y Abstraccts	3
2.1.7 Campos privados y públicos	3
2.1.8 Variables asociadas a objetos	4
2.1.9 Iteradores y Generadores	4
2.1.10 Encapsulación y visibilidad	4
2.1.11 Métodos asincrónicos.....	4
2.1.12 Subclases	4
2.1.13 Mixins y composición	4
2.1.14 Uso de instanceof y Object.create.....	5
2.1.15 Métodos especiales de objetos	5
2.1.16 Muestra del código	5
2.2 Problemas comunes	6
2.3 Similitud con la vida real.....	6
3.Ejemplos prácticos	6
Ejemplo 1:.....	6
Ejemplo 2:	6
Ejemplo a realizar 1:	6
Ejemplo a realizar 2:	6
4.Resumen	6

1.Título del tema

Introducción de la Programación Orientada a Objetos en JavaScript

2.Explicación detallada del contenido

La Programación Orientada a Objetos (POO) organiza el código alrededor de objetos que combinan datos (propiedades) y acciones (métodos). En JavaScript, la POO permite escribir programas más claros, reutilizables y fáciles de mantener, especialmente a medida que el proyecto crece.

- **Orientación a objetos:** Representa entidades de la vida real con propiedades y métodos.
- **Palabras clave:** class, constructor, new, extends, this.
- **Problemas comunes:** Código desordenado con funciones y variables sueltas.
- **Similitud con la vida real:** Modelar personas, coches, animales... cada uno con características y acciones.

2.1 Qué es la POO en JavaScript

La Programación Orientada a Objetos (POO) es un estilo de programación que utiliza estructuras llamadas objetos para representar elementos de la vida real.

JavaScript permite crear y usar objetos para organizar el código, dividir tareas y mantener todo ordenado.

2.1.1 Class

La palabra clave **class** se usa para crear un molde. Ese molde sirve para construir objetos. Ejemplo:

```
class Persona {  
  #dni; // campo privado  
}
```

2.1.2 Constructor

Dentro de una clase, el **constructor** es un bloque que se ejecuta cuando se crea un objeto que sirve para darle valores al nuevo objeto. Ejemplo:

```
class Persona {  
  // El constructor recibe parámetros y asigna valores al objeto  
  constructor(nombre, edad) {  
    this.nombre = nombre; // 'this' se refiere al objeto actual  
    this.edad = edad;      // Guardamos la edad en la propiedad del objeto  
  }  
}
```

2.1.3 Getters y Setters

Permiten controlar cómo se accede y modifica una propiedad.

```
get edad() {
  return this._edad;
}
set edad(valor) {
  if (valor < 0) throw new Error("La edad no puede ser negativa");
  this._edad = valor;
}
get dni() {
  return this.#dni;
}
```

2.1.4 Método

Un método es una función que describe una acción del objeto. Ejemplo:

```
saludar() {
  console.log("Hola, soy " + this.nombre);
}
```

2.1.5 Sobrescritura de métodos heredados

Se pueden definir métodos como toString y valueOf para comportamiento. Ejemplo:

```
toString() {
  return `Persona: ${this.nombre}, Edad: ${this.edad}`;
}
valueOf() {
  return this.edad; // devuelve la edad como valor numérico
}
```

2.1.6 Statics y Abstraccts

Los métodos estáticos pertenecen a la clase, no al objeto.

```
static esMayorDeEdad(edad) {
  return edad >= 18;
}
```

2.1.7 Campos privados y públicos

Los campos privados se declaran con # y no son accesibles desde fuera.

```
mostrarCampos() {
  console.log(`Nombre: ${this.nombre}, DNI: ${this.#dni}`);
}
```

2.1.8 Variables asociadas a objetos

Son las propiedades que cada objeto guarda.

```
asignarHobbies(hobbies) {  
  this.hobbies = hobbies;  
}
```

2.1.9 Iteradores y Generadores

Permiten recorrer colecciones dentro de la clase.

```
*[Symbol.iterator]() {  
  if (!this.hobbies) return;  
  for (const hobby of this.hobbies) yield hobby;  
}
```

2.1.10 Encapsulación y visibilidad

La encapsulación protege los datos internos de un objeto.

```
getDniSeguro() {  
  return `El DNI está protegido: ${this.#dni}`;  
}
```

2.1.11 Métodos asincrónicos

Los métodos asincrónicos permiten trabajar con operaciones que tardan tiempo, como llamadas a APIs.

```
async obtenerDatos() {  
  return new Promise(resolve => {  
    setTimeout(() => resolve(`Datos de ${this.nombre}`), 1000);  
  });  
}
```

2.1.12 Subclases

Una **subclase** es una clase que hereda de otra, que se hace con **extends**.

Ejemplo:

```
class Estudiante extends Persona {  
  estudiar() {  
    console.log(`${this.nombre} está estudiando`);  
  }  
}
```

Es como decir: "Estudiante es un tipo de Persona".

2.1.13 Mixins y composición

Se pueden añadir comportamientos extra con mixins.

```
const Hablador = Base => class extends Base {  
  hablar() { console.log("Estoy hablando..."); }  
};  
class PersonaHabladora extends Hablador(Persona) {}
```

2.1.14 Uso de instanceof y Object.create

- **instanceof** comprueba si un objeto pertenece a una clase.
- **Object.create** permite crear objetos basados en un prototipo.

```
const persona2 = new Persona("Lucía", 18, "98765432B");
console.log(persona2 instanceof Persona); // true
const proto = { hablar() { console.log("Guau"); } };
const perro = Object.create(proto);
perro.hablar(); // Guau
```

2.1.15 Métodos especiales de objetos

JavaScript ofrece métodos útiles para trabajar con objetos:

```
const obj = { a: 1 };
console.log(obj.hasOwnProperty("a")); // true
const destino = {};
Object.assign(destino, { x: 1 }, { y: 2 });
console.log(destino); // { x: 1, y: 2 }
const original = { nested: { v: 1 } };
const copia = structuredClone(original);
copia.nested.v = 2;
console.log(original.nested.v); // 1
```

2.1.16 Muestra del código

```
const persona1 = new Persona("Carlos", 28, "12345678A");
persona1.saludar(); // Hola, soy Carlos y tengo 28 años
console.log(persona1.dni); // 12345678A
console.log(persona1.toString()); // Persona: Carlos, Edad: 28
console.log(+persona1); // 28
console.log(Persona.esMayorDeEdad(20)); // true

persona1.mostrarCampos(); // Nombre: Carlos, DNI: 12345678A
persona1.asignarHobbies(["leer", "correr"]);
for (const hobby of persona1) console.log(hobby); // leer, correr
console.log(persona1.getDniSeguro()); // El DNI está protegido: 12345678A
persona1.obtenerDatos().then(datos => console.log(datos)); // Datos de Carlos
const estudiante1 = new Estudiante("Ana", 22, "11111111C");
estudiante1.saludar(); // Hola, soy Ana y tengo 22 años
estudiante1.estudiar(); // Ana está estudiando
const personaHabladora = new PersonaHabladora("Luis", 30, "22222222D");
personaHabladora.hablar(); // Estoy hablando...
```

2.2 Problemas comunes

Cuando no organizamos bien el código:

- Hay demasiadas variables sueltas
- Hay funciones por todos lados
- Es difícil encontrar dónde cambiar algo
- Todo se vuelve confuso

La POO ayuda a ordenar todo dentro de objetos y clases.

2.3 Similitud con la vida real

La POO funciona como en la vida real:

- Una Persona tiene nombre, edad y puede hablar
- Un Coche tiene marca, modelo y puede arrancar
- Un Animal tiene tipo y puede hacer sonidos

Cada cosa tiene características y acciones; así funcionan los objetos.

3. Ejemplos prácticos

Ejemplo 1:

Ejemplo 2:

Ejemplo a realizar 1:

Ejemplo a realizar 2:

4. Resumen

- **La POO** usa objetos para organizar el código.
- **Una clase** es un molde.
- **Un objeto** es algo creado con ese molde.
- El **constructor** inicializa los valores.
- **new** crea nuevos objetos.
- **Las subclases** permiten heredar características.

- Es útil para tener el código ordenado y fácil de entender.
-